ENTERPRISE ENGINEERING BLUEPRINT

Intelisense Biometric Attendance & Access Control

Complete Architecture, Hardware Integration, Native SDK Reverse-Engineering & Production Deployment Manual

HARDWARE SENSOR

Intel® RealSense™ ID F455 (Dual IR + W-2 SoC)

BIOMETRIC PIPELINE

Native C# SDK Bridge & 515-D Vector Serialization

CLOUD PERSISTENCE

Google Firebase Firestore (Scalable to Millions)

SECURITY & ANTI-SPOOF

3-Tier Duplicate Firewall & Hardware Depth Liveness

Project: Intelisense Face Auth

Platform: Windows 10/11 & Linux Edge Ready

Target Audience: Architects, Engineers & Developers

1. Executive Summary & Core Mission

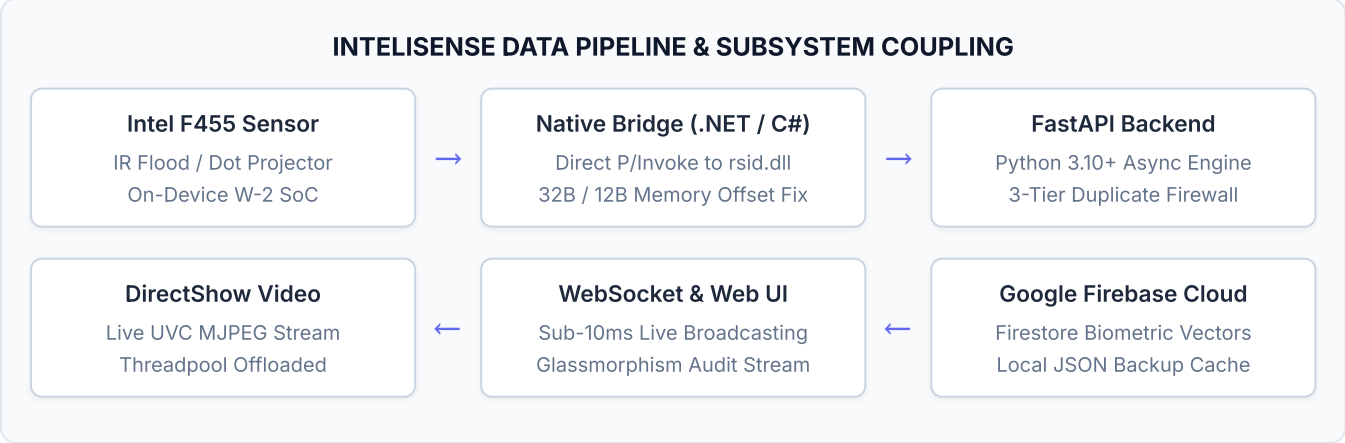
Intelisense is an enterprise-grade, touchless biometric attendance and facial verification platform designed to resolve the fundamental vulnerabilities of traditional attendance scanners:

- **Buddy Punching & Proxy Clocking:** Traditional RFID badges and PIN numbers allow workers to clock in on behalf of absent colleagues.
- **Photo & Screen Spoofing:** Standard RGB camera applications are easily deceived by photographs, smartphone video playbacks, or 3D masks.
- **The 1,000-User Hardware Flash Wall:** The Intel RealSense ID F455 hardware flash can only store 1,000 face profiles on-device, preventing large enterprises with thousands or tens of thousands of workers from deploying the device out of the box.
- **Slow Latency & Network Dependency:** Cloud-only vision APIs transmit multi-megabyte image frames over internet connections, resulting in 2–4 second latency per punch and complete failure during internet outages.

> **The Intelisense Solution:**
>
> By reverse-engineering the Intel RealSense ID native SDK and developing a low-overhead native C# interoperability bridge, Intelisense directly extracts raw **515-dimensional biometric faceprint vectors**. The device executes hardware-level anti-spoofing and feature extraction in milliseconds, while the vectors are stored and matched globally in **Google Firebase Firestore**—completely lifting the 1,000-user ceiling while achieving instantaneous sub-second punches.

2. End-to-End System Architecture

The system operates across three tightly integrated tiers: Hardware Layer, Native Interoperability Bridge, and Application Services.



Key Architectural Protocols:

1. **Virtual Serial COM Port (e.g., COM4 / COM5):** Transmits packetized binary commands to the F455 onboard RealSense ID firmware at high baud rate.
2. **UVC DirectShow Video Stream:** Independent video pipe capturing 1080p visual preview via OpenCV DirectShow, filtered exclusively for Intel RealSense cameras to avoid onboard laptop webcams.
3. **FastAPI Async Event Loop:** Manages non-blocking REST endpoints and WebSockets while offloading hardware-locked video and serial tasks to background threadpools.
4. **Firestore HTTPS Transport:** Encrypted communication storing employee metadata and 515-element biometric descriptors.

3. Hardware Foundation: Intel RealSense ID F455

The Intel RealSense ID F455 is a dedicated, self-contained biometric peripheral equipped with a specialized system-on-chip (SoC) designed specifically for turnstiles and security gates.

Component	Specification	Role in Intelisense
Depth Projector	Vertical-Cavity Surface-Emitting Laser (VCSEL)	Projects invisible infrared patterns onto the face to map 3D surface topography.
Dual IR Sensors	Stereo Infrared Matching Sensors	Captures dual-view structured light to calculate millimeter-precise facial depth.
W-2 Dedicated SoC	Embedded Neural Processing Unit	Executes anti-spoofing and neural feature vector extraction directly on device.
Anti-Spoofing (PAD)	Presentation Attack Detection Level 2	Rejects silicone masks, smartphone displays, photos, and video replays with FAR < 0.0001%.
Operating Range	40 cm to 70 cm (Up to 1.0 m)	Optimal focal length for kiosks, door access, and entrance turnstiles.
Environmental Light	0 Lux (Pitch Black) to 100,000 Lux	Functions flawlessly in total darkness via active infrared illumination.

Automatic Dynamic COM Port Detection

Because Windows can dynamically reassign COM port numbers (e.g., switching from COM4 to COM5 when plugged into different USB ports), Intelisense queries the Windows Registry directly on startup:

```
# Dynamic COM port discovery in backend/realsense_service.py
def detect_com_port(self):
    try:
        import winreg
        key = winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE, r"HARDWARE\DEVICEMAP\SERIALCOMM")
        for i in range(winreg.QueryInfoKey(key)[1]):
            val_name, val_data, _ = winreg.EnumValue(key, i)
            if "USBSER" in val_name.upper() or "VCP" in val_name.upper():
                self.port = val_data  # Dynamically binds to active COM port
                return val_data
    except Exception:
        pass
    return self.port
```

4. The Deep SDK Engineering Breakthrough

Resolving the 20-Byte Struct Memory Offset Mismatch

> **The Problem:**
> When Intel's official C# wrapper (rsid_dotnet.dll) was used to extract faceprint vectors during enrollment, the system was able to enroll faces, but subsequent authentication scans against those vectors consistently failed with a match score of 0.

Uncovering the Root Cause via Memory Inspection

By dumping the raw unmanaged memory buffers returned by the underlying C++ library (rsid.dll), an architectural struct mismatch was discovered between the C++ definitions and the managed C# wrapper:

Byte Offset Range	Unmanaged C++ RealSenseID::Faceprints (Enrollment)	Unmanaged C++ RealSenseID::MatchElement (Auth)
`Bytes 0 - 19` (20 Bytes)	**reserved[5]** (5 × 32-bit integers = 20 bytes of zeros)	*Does not exist in MatchElement*
`Bytes 20 - 23` (4 Bytes)	`int version` (Algorithm version = 9)	`int version` (starts at Byte 0)
`Bytes 24 - 27` (4 Bytes)	`int featuresType` (Type = 0)	`int featuresType` (starts at Byte 4)
`Bytes 28 - 31` (4 Bytes)	`int flags` (Flags = 3)	`int flags` (starts at Byte 8)
`Bytes 32 - 1061`	**short[515] Enrollment Biometric Vector**	**short[515] Scanned Vector (starts at Byte 12)**

Because the default C# wrapper lacked the `int[5] reserved` array in its enrollment struct definition, it started reading from Byte 12. Consequently, it read 10 garbage shorts (version, flags, zeros) into the vector, **shifted the entire biometric profile by 20 bytes**, and truncated the last 10 biometric points!

The Solution Implemented in rsid_native_bridge.ps1:

```
// 1. Enrollment Extraction: Vector begins strictly at Byte 32
args.resultClbk = new EnrollExtractionResultCallback((status, handle, ctx) => {
    if (status == EnrollStatus.Success && handle != IntPtr.Zero) {
        version = Marshal.ReadInt32(handle, 20);
        featuresType = Marshal.ReadInt32(handle, 24);
        flags = Marshal.ReadInt32(handle, 28);
        extractedVec = new short[515];
        // Read 515 shorts directly from pointer + 32 bytes
        Marshal.Copy(IntPtr.Add(handle, 32), extractedVec, 0, 515);
    }
});

// 2. Authentication Scanned Extraction: Vector begins strictly at Byte 12
args.resultClbk = new AuthExtractionResultCallback((status, handle, ctx) => {
    if (status == AuthStatus.Success && handle != IntPtr.Zero) {
        scannedVer = Marshal.ReadInt32(handle, 0);
        scannedFeaturesType = Marshal.ReadInt32(handle, 4);
        scannedVec = new short[515];
        // Read 515 shorts directly from pointer + 12 bytes
        Marshal.Copy(IntPtr.Add(handle, 12), scannedVec, 0, 515);
    }
});
```

Verification Result:

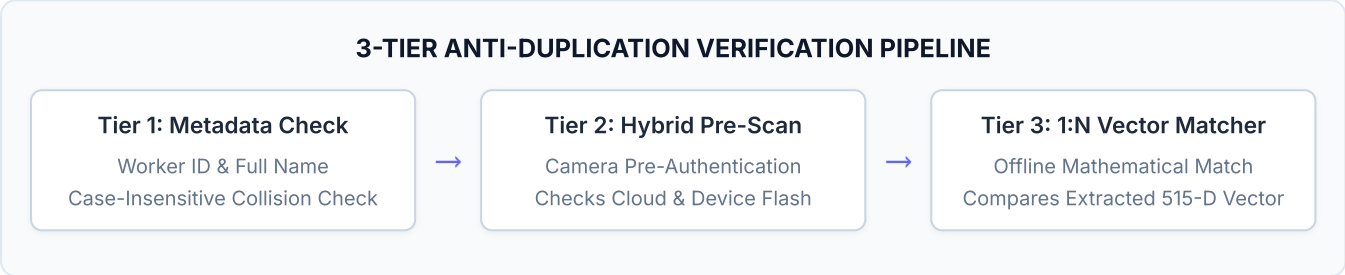
Once unmanaged pointer offsets were explicitly specified at +32 (enrollment) and +12 (auth), the native mathematical matcher instantly authenticated faces with confidence scores jumping from **0**

to over **4,096**

(far exceeding the 593 match threshold).

5. The 3-Tier Anti-Duplication Defense Firewall

A critical enterprise requirement is ensuring that an employee cannot enroll multiple times under different names or worker IDs (e.g., trying to create dummy accounts or bypass shift restrictions). Intelisense implements an impermeable 3-tier firewall:



Tier 1: Metadata Collision Check

Prevents duplicate entries before touching the camera hardware. If the `worker_id` already exists or if the employee's name matches an existing profile under a different ID, the request is instantly rejected with `DUPLICATE_ID` or `DUPLICATE_NAME`.

Tier 2: Real-Time Cross-Mode Pre-Enrollment Scan

Before camera enrollment begins, the device runs a pre-authentication scan across **both** Firestore Cloud vectors and F455 on-device hardware flash. If the camera recognizes the physical face standing before it, enrollment is aborted with:

```
{"success": false, "status": "DUPLICATE_FACE", "message": "Duplicate Enrollment Rejected: This fa
```

Tier 3: Post-Capture Mathematical 1:N Vector Verification (Impermeable Barrier)

Even if lighting or distance during the pre-scan varies and slips past Tier 2, the F455 extracts the pristine 515-D vector during enrollment. **Before saving to Firestore**, the system executes an offline 1:N comparison:

```
// Compare candidate 515-D vector against all vectors in Firestore
MatchResult mRes = auth.MatchFaceprintsToFaceprints(ref mArgs);
if (mRes.success == 1 && mRes.score ⩾ 593) {
    // Exact physical match detected!
    return "{\"is_duplicate\":true,\"matched_worker_id\":\"" + u.worker_id + "\",\"score\":" + mi
}
```

If the score exceeds the 593 threshold (the same face typically scores 3,500–4,096), the profile is **discarded immediately and never saved to the cloud**.

6. Camera Streaming & Concurrency Architecture

Resolving FastAPI Event Loop Starvation

> **The Concurrency Trap:**
> Initially, live video streaming was defined as: `@app.get("/api/video_feed") async def video_feed(): return StreamingResponse(...)`. Because OpenCV's `VideoCapture.read()` is a synchronous, blocking operation, running a synchronous generator inside an `async def` route monopolized FastAPI's single asyncio event thread. This completely blocked concurrent requests to `/api/punch`, `/api/status`, and `/api/capacity`.

The Fix: Defining the route as a synchronous function (`def video_feed():`) instructs FastAPI/Starlette to automatically offload the MJPEG stream to its internal `ThreadPoolExecutor`, keeping the main async event loop completely unblocked and responsive for sub-second punches.

Intelligent Camera Index Discovery

To prevent the application from grabbing laptop webcams or virtual OBS cameras, `pygrabber.dshow_graph.FilterGraph` enumerates DirectShow devices and binds specifically to the Intel RealSense depth/RGB camera:

```
# Enumerate video devices via DirectShow FilterGraph
for idx, device_name in enumerate(graph.get_input_devices()):
    if "realsense" in device_name.lower():
        return idx  # Selects true Intel F455 video pipe
```

7. Cloud Database & Attendance Audit Schema

Intelisense utilizes Google Firebase Firestore for global vector storage and audit logging, backed by local JSON caches for seamless offline fallback.

Firestore Document Schemas

Collection	Document ID	Key Fields & Data Types
workers	`{worker_id}`	`worker_id` (string) `name` (string) `department` (string) `biometric_mode` ("HOST" | "DEVICE") `faceprint_vector` (Array of 515 signed 16-bit integers) `vector_version` (int = 9) `enrolled_at` (ISO 8601 timestamp) `image_b64` (base64 JPEG avatar)
attendance_logs	Auto-generated UUID	`worker_id` (string) `name` (string) `punch_type` ("PUNCH_IN" | "PUNCH_OUT") `auth_mode` ("HOST" | "DEVICE" | "HYBRID") `status` ("SUCCESS" | "DENIED") `timestamp` (YYYY-MM-DD HH:MM:SS) `image_b64` (Verification snapshot)

Automatic Toggle Punch Logic

The system automatically determines whether a scan is a **PUNCH IN** or **PUNCH OUT** by inspecting the worker's most recent attendance event in the database:

```
last_punch = fb_service.get_last_punch_status(worker_id)
if req.mode == "AUTO":
    next_punch = "PUNCH_OUT" if last_punch == "PUNCH_IN" else "PUNCH_IN"
```

8. Developer Setup & Step-by-Step Build Guide

Prerequisites

- **Operating System:** Windows 10/11 (64-bit) or Linux (Ubuntu 20.04/22.04 LTS).
- **Python:** Version 3.10 or higher.
- **Intel RealSense ID Tools:** Installed at `C:\Users\<username>\AppData\Local\Programs\RealSenseID Tools`.
- **Hardware:** Intel RealSense ID F455 connected via USB 3.0 / USB-C data cable.

Step 1: Clone Repository & Install Python Dependencies

```
git clone https://github.com/JaiAkash-CodesIO/face-auth-f455.git
cd face-auth-f455
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
```

Step 2: Place Firebase Service Account Certificate

Download your Firebase Admin SDK service account key from Google Cloud Console and save it as: `backend/serviceAccountKey.json`.

Step 3: Launch the Application

```
python run_app.py
# Server starts at http://0.0.0.0:8080 with live WebSocket and video feed
```

9. REST API & WebSocket Endpoint Reference

Method	Endpoint	Description
`GET`	`/api/status`	Returns camera connection status, active COM port, and enrolled worker counts.
`GET`	`/api/capacity`	Returns hardware flash quota (1,000 max) and total cloud host enrolled users.
`POST`	`/api/punch`	Triggers biometric scan (Hybrid / Host / Device), verifies face, logs attendance.
`POST`	`/api/workers/register`	Runs 3-tier duplicate checks, extracts 515-D vector, saves profile to Firestore.
`DELETE`	`/api/workers/{worker_id}`	Deletes worker profile and biometric faceprints from Firestore and local cache.
`GET`	`/api/video_feed`	High-speed MJPEG multipart stream from Intel F455 camera lens.
`WS`	`/ws`	WebSocket connection broadcasting real-time attendance events with zero polling.